

UE25CS251A
DIGITAL LOGIC AND COMPUTER DESIGN

TEAM DLCD

Department of CSE and CSE (AI & ML)

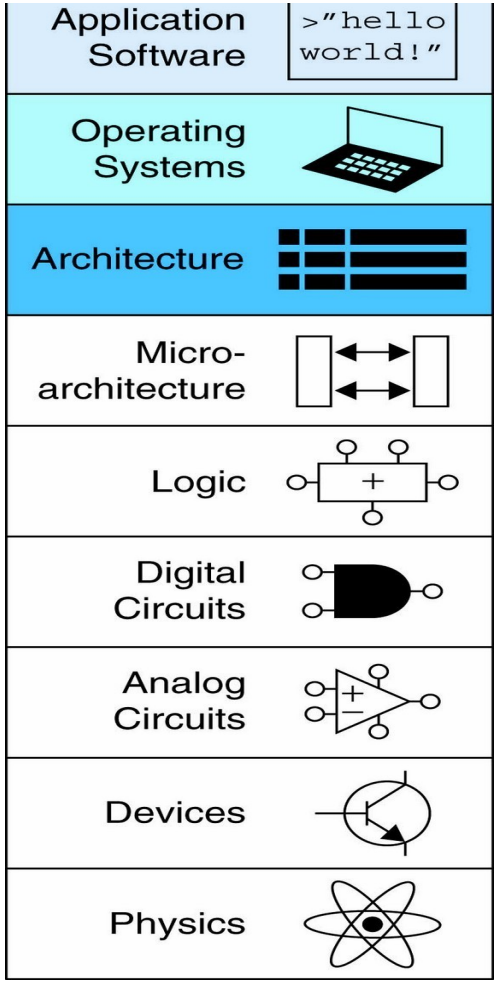
DIGITAL LOGIC AND COMPUTER DESIGN

ARCHITECTURE-Addressing Modes

TEAM DLCD
Department of CSE and CSE (AI & ML)

Addressing Modes

- **Addressing Modes**



Addressing Modes

What is an Addressing Mode?

An **addressing mode** specifies **how the CPU locates the operand(s)** required to execute an instruction. Depending on the instruction, operands may be stored in **registers, memory locations, or within the instruction itself**.

Why are Addressing Modes Important?

- Determine **where operands are located**
- Influence **execution speed and memory usage**
- Provide flexibility for handling constants, variables, arrays, loops, and function calls
- Reduce instruction size while improving processor efficiency

Addressing Modes

MIPS Addressing Modes:

- **Register Only**
  - Operands are stored entirely in **CPU registers**.
  - Used for arithmetic and logical operations.
  - **Example:** `add $s0, $t2, $t3`
- **Immediate**
  - One operand is a **constant value** encoded directly within the instruction.
  - Eliminates the need to fetch the constant from memory.
  - **Example:** `addi $s0, $s0, 10`
- **Base Addressing**
  - The operand is stored in **memory**.
  - The effective address is calculated using a **base register and an offset**.
  - **Example:** `lw $t0, 8($sp)`

Addressing Modes

MIPS Addressing Modes:

- **PC-Relative Addressing**
  - Used by **branch instructions**.
  - The destination address is calculated relative to the **Program Counter (PC)**.
  - **Example:** beq $t0, $0, Loop
- **Pseudo Direct Addressing**
  - Used by **jump instructions**.
  - The instruction specifies the jump target address to transfer program control.
  - **Example:** j main

Key Formulae:

Base Addressing: Effective Address = Base Register + Offset

PC–Relative Addressing: Branch Target Address = PC + 4 + (Offset × 4)

Addressing Modes

Register-Only Addressing:

- **Definition:** Register-only addressing uses **CPU registers** for both the source and destination operands. No memory locations are accessed during instruction execution.
- **Used By:** All **R-type instructions** in the MIPS architecture.
- **Operation:** The ALU reads operands directly from the register file, performs the required operation, and stores the result back into a register.
- **Advantage:** Since registers are the fastest storage elements in the processor, register-only addressing provides **high execution speed** and minimizes memory access.
- **Example:** `add $s0, $t2, $t3`
  - Source operands: `$t2`, `$t3`
  - Destination operand: `$s0`

Key Point:

All operands remain within the CPU registers, making Register-Only Addressing the fastest addressing mode in MIPS.

Addressing Modes

Register-Only Addressing:

- **Operands are stored entirely in CPU registers.**
- The instruction reads the required operands from the register file, performs the operation in the ALU, and stores the result back into a register.
- **No memory access** is required during execution, making this addressing mode fast and efficient.

Examples:

- `add $s0, $t2, $t3`
  - Performs: **$s0 = $t2 + $t3**
- `sub $t8, $s1, $0`
  - Performs: **$t8 = $s1 – $0** (copying the value of `$s1` since `$0` always contains 0)

Note: Register `$0` is hardwired to the value **0** in MIPS and cannot be modified.

Addressing Modes

Immediate Addressing:

- **Definition:** Immediate addressing uses a **16-bit constant (immediate value)** encoded directly within the instruction, along with a register operand.
- **Instruction Format:** One operand is a register, while the other is the immediate value specified in the instruction.
- **Used By:** I-type instructions such as **addi (Add Immediate)**, **ori (OR Immediate)**, **andi (AND Immediate)**, and **lui (Load Upper Immediate)**.
- **Advantage:** Since the constant is part of the instruction, the processor does **not need to access memory**, resulting in faster execution.
- **Applications:** Commonly used for initializing registers, updating loop counters, incrementing/decrementing values, and performing arithmetic or logical operations with constants.

Key Point:

Immediate addressing combines a register operand with a constant value embedded in the instruction, making it efficient for operations involving fixed constants.

Addressing Modes

Immediate Addressing:

Examples: `addi $s4, $t5, -73`

- Performs: **$s4 = $t5 + (-73)**
- Adds the immediate value **−73** to the contents of register **$t5** and stores the result in **$s4**.

`ori $t3, $t7, 0xFF`

- Performs: **$t3 = $t7 OR 0x00FF**
- Performs a bitwise OR between the contents of **$t7** and the hexadecimal constant **0xFF**, storing the result in **$t3**.

Key Point:

Immediate addressing is ideal for operations involving fixed constants, as the operand is embedded directly within the instruction, resulting in faster execution.

Addressing Modes

Base Addressing:

- **Definition:** Base addressing is used to access data stored in **memory**. The memory address is calculated by adding a **base address** stored in a register to a **16-bit signed offset** (immediate value).
- **Used By:** Memory access instructions such as **lw (Load Word)** and **sw (Store Word)**.
- **Effective Address Calculation:**

  Effective Address = Base Register + Sign-Extended Offset
- **Operation:** The processor first computes the effective address, then reads data from memory (lw) or writes data to memory (sw).
- **Applications:** Commonly used for accessing arrays, stack variables, structures, and dynamically allocated memory.

Key Point:

Base addressing enables efficient memory access by combining a register-based base address with a signed offset, allowing programs to access different memory locations using the same instruction format.

Addressing Modes

Base Addressing:

- The memory address is obtained by adding the **base register** to the **sign-extended 16-bit offset**.
- The calculated value is called the **Effective Address (EA)**.

Examples: `lw $s4, 72($0)`

- Effective Address = $0 + 72
- Since **$0 = 0**, the address becomes **72**.
- Loads the word stored at memory address **72** into register **$s4**.

`sw $t2, -25($t1)`

- Effective Address = $t1 + (-25)
- Stores the contents of register **$t2** into the memory location **25 bytes before** the address contained in **$t1**.

Note: The offset may be **positive or negative**, allowing access to memory locations before or after the base address. This addressing mode is widely used for arrays, stack frames, and structures.

Addressing Modes

PC-Relative Addressing:

- **Definition:** PC-relative addressing is used by **conditional branch instructions** to determine the next instruction to execute when a branch condition is satisfied.
- Instead of storing the complete destination address, the instruction stores a **16-bit signed offset** relative to the current **Program Counter (PC)**.
- The processor computes the **Branch Target Address (BTA)** using the formula:

$$BTA=(PC+4)+(\text{Sign-Extended Offset}\times 4)$$

- **Used By:** Branch instructions such as **`beq`** (Branch if Equal) and **`bne`** (Branch if Not Equal).
- **Advantage:** Reduces instruction size while allowing both forward and backward branching within a program.

Key Point: PC-relative addressing specifies the destination relative to the current instruction rather than storing the complete address.

Addressing Modes

PC-Relative Addressing – Example

Assembly Code:

```
0x10   beq  $t0, $0, else
0x14   addi $v0, $0, 1
0x18   addi $sp, $sp, 8
0x1C   jr   $ra
0x20 else: addi $a0, $a0, -1
0x24       j    factorial
```

Machine Code Fields:

- **Opcode (op)** = 4 (beq)
- **Source Register (rs)** = $t0 = 8
- **Target Register (rt)** = $0 = 0
- **Immediate (offset)** = 3

Thus, the instruction is encoded as: beq $t0, $0, 3

Addressing Modes

Offset Calculation:

- Current PC = **0x10**
- Next instruction = **PC + 4 = 0x14**
- Branch target (else) = **0x20**
- Distance = **3 instructions**

Therefore, Immediate Offset = 3

Key Observation:

- The immediate field stores the **number of instructions** to jump, **not the actual memory address**.
- During execution, the processor **sign-extends the offset**, multiplies it by **4** (word addressing), and adds it to **PC + 4** to compute the branch target address.

Addressing Modes

Pseudo- Direct Addressing:

- **Definition:** Pseudo-direct addressing is used by **jump instructions** to transfer program control to another location in the program.
- The jump instructions **`j` (Jump)** and **`jal` (Jump and Link)** use a **26-bit address field** instead of storing the complete 32-bit target address.
- The processor constructs the **32-bit Jump Target Address (JTA)** by combining:
  - The **upper 4 bits** of **PC + 4**, and
  - The **26-bit address field** shifted left by **2 bits**.
- This addressing mode is called **pseudo-direct** because only part of the destination address is stored in the instruction.

Jump Target Address Formula

$$JTA=\{(PC+4)_{31:28}, Address_{25:0}, 00\}$$

Addressing Modes

Pseudo-Direct Addressing:

Used By

- `j` (Jump)
- `jal` (Jump and Link)

Key Point: Pseudo-direct addressing stores only a 26-bit target field, reducing instruction size while allowing efficient jumps within the same 256 MB memory region.

Addressing Modes

Pseudo-Direct Addressing – Example

Assembly Code

```
0x0040005C      j   sum

...

0x004000A0  sum:
                    add  $v0, $a0, $a1
```

Address Calculation

- Jump Target Address (JTA) = **0x004000A0**
- 26-bit Address Field = **0x0100028**
- Opcode for `j` = **000010**

Addressing Modes

During execution, the processor:

1. Takes the **upper 4 bits of PC + 4**
2. Appends the **26-bit address field**
3. Adds **two trailing 0 bits** (word alignment)
4. Forms the complete **32-bit Jump Target Address**

Machine Code Format

Opcode (6 bits) | Address (26 bits)

000010 | 00000100000000000000101000

Machine Code:

0x08100028

Addressing Modes

Why Shift Left by 2?

- Every MIPS instruction is **4 bytes (32 bits)** long.
- Instruction addresses are therefore **word-aligned**, so the last **2 bits are always 00**.
- These bits are omitted from the instruction and restored by the processor during execution.

Key Observation:

Unlike PC-relative addressing, pseudo-direct addressing does not store an offset. Instead, it stores part of the destination address, which the processor combines with the upper bits of the Program Counter to obtain the full jump target.

MCQ

Q1. Register-only addressing is primarily used by which type of MIPS instructions?

A) I-type instructions
B) J-type instructions
C) R-type instructions
D) Memory access instructions

MCQ

Q2. In immediate addressing, the constant operand is:

A) Stored in memory
B) Stored in a register
C) Encoded within the instruction itself
D) Retrieved from the stack

MCQ

Q3. Which MIPS instructions use base addressing?

A) add and sub

B) beq and bne

C) lw and sw

D) j and jal

MCQ

Q4. In base addressing, the effective memory address is calculated as:

A) PC + Immediate

B) Base Register + Sign-Extended Offset

C) Register + Register

D) PC + Jump Target Address

MCQ

Q5. PC-relative addressing is mainly used by:

A) Arithmetic instructions

B) Load and store instructions

C) Conditional branch instructions

D) Jump instructions

MCQ

Q6. In PC-relative addressing, the branch target address is calculated relative to:

A) Current PC

B) PC + 4

C) Stack Pointer ($sp)

D) Global Pointer ($gp)

MCQ

Q7. Which addressing mode is used by the j and jal instructions?

A) Register-only Addressing

B) Immediate Addressing

C) Base Addressing

D) Pseudo-direct Addressing

MCQ

Answer Key

Q1: C **Q2: C** **Q3: C** **Q4: B** **Q5: C** **Q6: B** **Q7: D**



THANK YOU

TEAM DLCD

Department of CSE and CSE(AI & ML)